

WayLoom

The connected delivery planning and intelligent capacity platform

COMPLETE PRODUCT, DESIGNATHON, HACKATHON & DATATHON PLAN

A source-grounded, team-shareable implementation guide for the Waypoint Group challenge. Written in clear English, with official obligations separated from our recommended product decisions.

THE BUSINESS	THE PRODUCT	THE INTELLIGENCE
120 outlets · 3 retail brands · 2 depots · 60 vehicles	One workflow from order to loading, delivery and store receipt	Predict service time and lateness; forecast 10 weeks of demand; allocate a peak-day fleet

READING KEY

REQUIRED = explicit in the official challenge booklet. PROPOSED = our WayLoom design or engineering recommendation.
OPTIONAL = a further enhancement that must not displace required scope.

Document owner: WayLoom team | Edition: 28 September 2026 | Reference: official 33-page Rootcode Tech-Triathlon 2026 Challenge Booklet.
Internal planning document. All numerical worked examples in this plan are illustrative unless explicitly identified as official booklet examples. Do not publish supplied competition datasets or derivatives.

Document map

Use this document as the shared agreement between design, engineering and data science. Read Sections 1–7 together; then each workflow should use its detailed section.

PART	WHAT IT COVERS
A · Business and product	01 Snapshot and scope · 02 Real problem · 03 Required roles · 04 Full operational workflow
B · The build	05 Feature inventory · 06 Rules and exceptions · 07 Screens and UX · 08 Data model and states · 09 Architecture and technology
C · Competition workflows	10 Designathon · 11 Hackathon · 12 Datathon Task 1 · 13 Task 2A · 14 Task 2B
D · Delivery and quality	15 Dataset map and data safety · 16 Integration · 17 Timeline · 18 QA and demo · 19 Risks · 20 Team sign-off
Appendices	A Acceptance checklist · B Working glossary · C Official-source map

Decisions that should not be misunderstood

- The four official user roles are Dispatcher, Loader, Driver and Store Manager. Administrator is NOT an official fifth operational role.
- The Hackathon build follows the team's submitted Designathon work. Significant departures need to be documented in the README.
- The Datathon is scored separately; integrating the trained models into the Hackathon web app is NOT mandatory.
- Task 2B has an explicitly defined scenario and seven feasibility rules. Do not silently substitute the Hackathon's wider operating rules for that scenario.
- Do not promise GPS tracking, artificial intelligence explanations or third-party integrations without real supporting data and implementation.

THE PRODUCT STORY IN ONE SENTENCE

WayLoom connects orders, responsible people and delivery events so Waypoint can allocate scarce vehicles fairly, keep field teams informed, record what really happened and plan ahead using verified data.

A | Business and product

01 Project snapshot

FACT	OFFICIAL POSITION
Theme	The Intelligent Enterprise. One fictional Waypoint Group delivery-planning challenge across three phases.
Brands / outlets	Waypoint Fresh: 80; Waypoint Style: 25; Waypoint Tech: 15. Total: 120 outlets.
Depots	Peliyagoda distribution centre and Kandy regional hub.
Fleet	60 vehicles: 12 reefer trucks, 40 dry-box trucks, 8 vans (4 vans reefer). Thus 16 vehicles can carry chilled goods.
Operating days	Monday–Saturday; use calendar.csv for actual operating dates.
Order cutoff	Orders for the next day close at 4 PM. Later orders wait until the following run.
Trip rules	At most two vehicle routes/trips per day; capacity must satisfy weight and volume; temperature, access, delivery windows and fuel quotas matter.
Competition data	Synthetic records shared across phases, including 120 outlets, 60 vehicles, two depots and common calendar.

OFFICIAL SOURCE · Booklet pp. 3–7 and 24–31.

02 Problem, vision and outcomes

The three brands compete for one finite vehicle fleet. Fresh needs early arrivals and refrigerated capacity; Style often fills volume before weight; Tech can be heavy, fragile and unpredictable. Some shops have fixed mall windows or van-only access. The current process depends on spreadsheets, phone calls, printed run sheets and a dispatcher's memory.

PAIN POINT	WAYLOOM RESPONSE	EVIDENCE OF SUCCESS
Orders are fragmented	Capture and confirm orders in one queue with an enforced cutoff.	One source of truth; order status visible to the store.
Plans can violate constraints	Validate capacity, temperature, access, time windows and fuel allowance.	Invalid assignments are rejected and explained.
Deferrals are hard to explain	Keep a reason, history and clear notification for every deferred order.	Dispatcher can show which outlets were skipped previously.
Load instructions go stale	Give Loader the current trip and stop sequence; allow shortage/damage flags.	Pre-departure exceptions reach planning.
Drivers lose coverage	Mobile-responsive workflow with offline capture and safe synchronization.	Proof of delivery remains available after reconnecting.
Demand is hard to anticipate	Independent Datathon forecasts and timing/risk predictions.	Documented test predictions and feasibility-tested allocation.

Product objectives (proposed measures)

- Operational correctness: no served order violates the validation rules in the demonstrated workflow.
- Traceability: a judge can follow one order from store submission through allocation, loading, driver outcome and receipt.
- Continuity: a driver can record a stop without coverage and see whether that update is queued or synchronized.
- Decision clarity: show why an order is deferred and expose the history of previous deferrals.
- Prediction reliability: demonstrate honest validation, correct label construction and reproducible final outputs.

03 Required users and role-specific needs

OFFICIAL ROLE	WORKING CONTEXT	WHAT WAYLOOM MUST LET THEM DO
Dispatcher	Large screen, stable connection at Peliyagoda office.	Review confirmed orders, plan vehicles and trips, identify deferrals, monitor progress and explain decisions.
Loader	Shared tablet/terminal at Peliyagoda or Kandy dock.	View current sequence; check loads; flag shortages or damage before departure.
Driver	Personal phone on the road; coverage can drop. Interactions while safely stopped.	See route and stops, record outcome and proof of delivery offline, synchronize later.
Store Manager	Desktop or phone at retail outlet.	Place/confirm orders, see ETA and deferral, confirm received goods and report issues.

OFFICIAL SOURCE · Booklet p. 6; all four role experiences must connect (pp. 7, 9, 12).

Access model — proposed implementation

Provide four seeded judge accounts, one per official role. Each role sees only its own actions and records. Internal user creation/admin configuration can be a backend/seed function; do not replace an official role with “Administrator.” Enforce authorization on API routes, not only by hiding buttons.

04 One end-to-end operational journey

STAGE	OWNER	SYSTEM EVENT / HANDOFF
1 · Place order	Store Manager	Create and confirm before 4 PM cutoff; record requested delivery day; acknowledge order.
2 · Close / queue	Dispatcher	Consolidate confirmed orders; retain late orders for following run.
3 · Allocate	Dispatcher	Assign served orders to compatible vehicle/trip; defer unserved ones with reason and history.
4 · Load	Loader	See current stop sequence; confirm prepared load or flag missing/damaged goods.
5 · Deliver	Driver	Access phone-sized run sheet; record arrival, outcome, issue and proof of delivery; queue while offline.
6 · Receive	Store Manager	See expected arrival/updates, confirm what arrived, report mismatches.

STAGE	OWNER	SYSTEM EVENT / HANDOFF
7 • Plan ahead	Dispatcher / Datathon	Use predicted time/risk and future demand estimates when relevant; Datathon integration is optional.

OFFICIAL SOURCE • Booklet p. 7. Note: Fresh can place separate dry and chilled orders for the same delivery day (p. 4).

SIMPLE STORY

Store OUT001 orders chilled dairy and dry groceries for the same date. They remain separate order records. The dispatcher plans compatible capacity; the loader flags any shortfall; the driver records proof of delivery even without signal; the store confirms actual goods. Historical records can support predictions, but predictions never replace actual events.

B | Detailed product and engineering plan

05 Full Wayloom feature inventory

The “Required” items below implement or directly support official scope. “Proposed” items specify our preferred interaction or technical method. “Optional” ideas are presentation differentiators only when the mandatory journey already works.

5.1 Shared system foundation

FEATURE	SUBFEATURES / ACCEPTANCE	STATUS
Four-role access	Four seeded accounts, correct landing pages, role-sensitive navigation; backend permission checks.	Required outcome; proposed RBAC method
Unified reference data	Seed shared outlet, vehicle, depot and calendar records consistently across phases.	Required
Records and state changes	Keep unique order/vehicle/trip IDs, linked events and auditable status updates.	Proposed essential architecture
Responsive UI	Desktop/phone layout; specifically verify Driver and Loader at phone size.	Required
Offline and recovery	Driver can complete core field records without connection, see unsynced status and reconcile on return.	Required capability
Clear validation and errors	Explain why a vehicle/order pairing or submission is invalid; preserve work when errors occur.	Proposed quality method

5.2 Store Manager experience

MAIN FEATURE	DETAILED SUBFEATURES / BUSINESS RULE
Order placement	Select products/requirements, volume and date as supported by design; save, review and confirm; show 4 PM next-day cutoff and acknowledgment.
Multiple order handling	Fresh can have ambient and chilled orders for one delivery day; retain distinct IDs and temperature requirements.
Order status & ETA	See confirmed / queued / scheduled / deferred; show available expected arrival information, not fabricated live GPS.
Deferral information	Clear reason and next-run implications, with appropriate status notice.
Receipt and issues	Confirm delivery, log actual received quantity/outcome and note shortage/damage or other issue.
Order history	Track own outlet's prior orders and resolutions (recommended extension).

5.3 Dispatcher experience

MAIN FEATURE	DETAILED SUBFEATURES / BUSINESS RULE
Confirmed order queue	View next-run orders, cutoff, brand, depot, delivery window, temp and load size; search/filter/sort.

MAIN FEATURE	DETAILED SUBFEATURES / BUSINESS RULE
Fleet visibility	Show vehicle type, home depot, reefer/ambient, weight/volume capacity, fuel allowance and assignment.
Plan and allocate	Manual with validation, assisted or automated planning permitted; assign orders to vehicle and trip, obey official operating constraints.
Deferrals	Explicit served/deferred outcome for excess demand; log reason and repeated skipped-outlet history.
Route/trip overview	Stop sequence and planned times; communicate current instruction to loaders/drivers.
Progress and exceptions	See loading shortfall, delivery status, receipt outcome and road/field issue reports as entered.
Future capacity view	Design a place for Task 2A forecasts and Task 1 risk insight if integrated later (optional).

5.4 Loader experience

MAIN FEATURE	DETAILED SUBFEATURES / BUSINESS RULE
Assigned loading runs	Select depot, vehicle and trip; view latest plan and version.
Load order	List goods/stops in a sequence that supports unloading; avoid stale printed list.
Quantity verification	Record what was actually loaded; flag missing or damaged items before departure.
Pre-departure handoff	Mark checked/exception; make the shortfall visible to Dispatcher and Driver.
Tablet ergonomics	Large tap targets, clear contrast, minimal typing, strong empty/error states.

5.5 Driver experience

MAIN FEATURE	DETAILED SUBFEATURES / BUSINESS RULE
My route	Phone-sized run sheet, route and stop sequence, key outlet window/access information.
Outcome capture	Record stop arrival and delivery outcome; note failed attempt/problem and proof of delivery.
Offline queue	Continue capturing events while offline; show “saved locally”, “pending sync” and “synced” distinctly.
Reconciliation	Replay queued events safely on reconnect; prevent duplicate submissions and handle conflicts.
Safety-focused UX	No complex interactions while vehicle is moving; actions intended while safely stopped.

5.6 Proposed differentiators — only after core scope

IDEA	ACTUAL VALUE	GUARDRAIL
Delivery Passport	One traceable order timeline: confirmed, allocated, loaded, dispatched, outcome, receipt.	Only recorded events; do not invent GPS or completed steps.
Discrepancy Detective	Guided human review of difference between expected/loaded/received quantities.	Suggested checks, not AI accusations or unsupported root causes.
Smart Receiving Assistant	Step-by-step received-quantity entry with instant mismatch summary.	Must still capture actual physical receipt.
Attention Centre	Show actionable pending confirmations, loading exceptions and repeated deferrals.	Use explicit rules, not unexplained “AI urgency”.
Forecast Panel	Show predicted demand and uncertainty in Dispatcher planning.	Optional; Datathon need not be integrated.
Scan-to-find product	Barcode-assisted search on appropriate devices.	Only when dataset/product identifiers and device support exist.

06 Operating constraints, edge cases and degradation

6.1 Shared real-world planning rules (official challenge)

DOMAIN	REQUIRED RULE
Weight & volume	Every assigned vehicle load must satisfy both capacity limits, not just weight.
Temperature	Only reefer vehicles carry chilled/frozen goods; reefer vehicles may also carry ambient goods.
Access	van_only outlets require vans; mall outlets have fixed delivery access windows.
Time windows	All outlets have delivery windows; Fresh requires arrival before opening at 8 AM, with individual windows possibly different.
Fuel quota	Weekly fuel usage depends on route distance and vehicle fuel profile.
Depot & operating day	Vehicles serve their assigned depot; at most two routes/trips each day; Monday–Saturday operation per calendar.
Demand overflow	When capacity is insufficient, explain what was deferred and its consequence; preserve reason/history.
Network gaps	Field work must remain usable offline and reconcile when service returns.

OFFICIAL SOURCE · Booklet pp. 4–5 and Hackathon p. 12. Task 2B has its own exact scenario validation (Section 14).

6.2 At least one fully designed degradation screen (Designathon requirement)

SCENARIO	WHAT THE SCREEN MUST EXPLAIN	RECOVERY
Recommended flagship: Driver loses coverage	Local save state, queued events, no false “submitted” signal.	Sync and reconciliation after reconnection.
Loader finds a shortfall	Expected vs physically available goods and dispatch impact.	Escalate/update plan before departure.
Vehicle cannot fit assigned goods	Specific weight/volume/temperature/access reason.	Select compatible alternative or defer with explanation.
Store delivery deferred again	History of missed run and notice to outlet.	Dispatcher reviews priority and records a reason.
Receipt mismatch	Expected, loaded if known, actual received; require confirmation.	Open exception record; preserve raw evidence.

The team needs one fully developed failure screen with a short “why it matters” paragraph; extra screens should only be included when response quality stays high. [Booklet p. 9]

07 Information architecture and screen plan (proposed)

ROLE	CORE SCREENS	WHY THESE SCREENS EXIST
Dispatcher	Dashboard; Order Queue; Plan/Allocate; Fleet; Routes/Trip Detail; Exceptions/Deferrals.	Complete planning and monitoring without having to infer constraints or missing records.
Loader	Assigned Runs; Load Detail/Stop Order; Shortfall Report; Load Handoff.	Current loading list and immediate pre-departure feedback.
Driver	My Run; Stop Detail; Record Outcome/PoD; Offline Sync Status.	Fast field actions on phone, with clear confirmation and reconnection.
Store Manager	Place/Confirm Order; My Orders; ETA/Deferral Detail; Confirm Receipt/Issue.	Confidence that order was received, when delivery is expected, and how to report the outcome.

Interaction principles

- Make the next valid action obvious; show what users can actually do, not decorative controls.
- State labels should describe verified data: “planned”, “saved locally”, “synced”, “confirmed” are different things.
- Use a consistent WayLoom component library but design different densities for dispatcher desktop and field phones.
- Put reasons beside deferrals and validation errors; do not force people to hunt through unrelated pages.
- Respect permission and privacy boundaries; avoid exposing another store’s orders.

Visual identity — proposed, not an official requirement

TOKEN	HEX	SUGGESTED USE
Deep navy	#19354A	Primary text, headings, sidebar and strong structural elements.
Teal	#087E80	WayLoom brand highlight, primary actions and connected-state emphasis.
Warm amber	#EFB750	Attention states: pending, at-risk, requires confirmation.
Soft coral	#DB7057	Destructive actions and genuine exception states.
Light backgrounds	#EDF7F6 / #F7F9FB	Comfortable cards, dense table grouping and quiet surfaces.

Use colour with text labels and icons, never as the only meaning carrier. Role identity may use subtle navigation accents without changing the meaning of status colours.

08 Data model and state rules (proposed implementation)

ENTITY	CORE FIELDS / RELATIONSHIP
User / Role	user_id, official role, account, active; restrict access to own assignment/depot/outlet.
Outlet	outlet_id, brand, district, depot, dock, van-only/mall access, window.
Vehicle	vehicle_id, depot, type, reefer/ambient, kg/m ³ capacity, fuel profile and quota.
Order / Order Item	order_id/delivery_id, outlet, requested date, temp, units, weight, volume, confirmed/cutoff status.
Plan / Trip / Stop	dispatch date, vehicle_id, trip 1/2, stop sequence, planned timing and allocation.
Deferral	order key, decision, reason, decision time, previous skips and notice state.
Loading Check	trip/stop, expected/loaded quantity, shortage/damage and loader event.
Delivery Event / PoD	stop, recorded time, outcome, issue, proof reference, device sync identifier.
Receipt / Discrepancy	store confirmation, actual quantity, difference, issue and resolution history.
Sync Event / Audit	event ID, originating device, version, local timestamp, server timestamp and conflict status.

Recommended state definitions

Order: Draft → Confirmed → Queued → Allocated OR Deferred. Deferred orders retain their original requested date and are reconsidered under defined rules.

Trip: Planned → Ready to Load → Loading Verified/Exception → Dispatched → Completed / Needs Follow-up.

Stop: Pending → Arrived → Delivered / Failed Attempt → Receipt Confirmed / Issue Reported (where applicable).

Offline record: Saved on device → Pending synchronization → Synced OR Conflict requiring review.

These state names are implementation recommendations; the official booklet defines workflows and outcomes rather than prescribing this exact schema. Preserve original order demand and event history instead of overwriting facts with latest predictions.

09 Architecture and recommended tech stack

Use a simple deployable monorepo; technology choices below are proposed rather than mandated. Optimize for reliability, clear code, repeatable setup and a judge walkthrough.

LAYER	SUGGESTED TOOLS	JOB
Product design	Figma	Four-role flows, high-fidelity prototype, degradation screen and handoff.
Responsive frontend	React + TypeScript + Vite + Tailwind	Shared design system, role views and phone-sized field screens.
Server and API	Node.js + Express (or equivalent team skill)	Business rules, permissions, planning decisions, validation and event endpoints.
Database	PostgreSQL + Prisma (or equivalent)	Consistent relational records and migrations.
Offline capture	Service worker + IndexedDB + event queue	On-device driver event collection, retry and reconciliation.
Data science	Python + Pandas + NumPy + scikit-learn	Labels, features, local model training, evaluation, CSV outputs.

LAYER	SUGGESTED TOOLS	JOB
Allocation	Plain Python / OR-Tools if justified	Produce a feasible Task 2B allocation with independent checks.
Run and delivery	Docker Compose + .env.example; hosted build	One-command stack incl. DB and seed; public deployed application.
Quality	Pytest / app tests / API tests / CI	Repeatable acceptance, authorization and data rule checks.

High-level component flow

Responsive role interface → authenticated API → planning/validation + operational event handling → relational database. A driver offline queue sends versioned delivery events when connectivity returns. The Datathon notebook reads the official private datasets independently and produces saved models, predicted CSVs and a peak-day allocation; connection to the web app is optional.

DO NOT CONFUSE A ROUTE MAP WITH LIVE TRACKING

Without verified GPS inputs, show planned stops and recorded status only. Do not design a moving vehicle marker that claims live location from nonexistent telemetry.

C | Phase-by-phase competition requirements

10 Designathon — what the design team must deliver

OFFICIAL SOURCE · Deadline: Tue 29 Sep 2026, 11:59 PM SLST. Booklet pp. 9–10.

OFFICIAL DELIVERABLE	WHAT TO INCLUDE IN WAYLOOM
User personas	One grounded persona each for Dispatcher, Loader, Driver and Store Manager.
Screen flows + rationale	Screens for all four roles; one paragraph per screen explaining purpose and priority.
Degradation design	At least one fully designed breakdown scenario, named and justified.
High-fidelity prototype	Usable cross-role journey, including important states and interactions.
Unlisted demo video	3–5 minutes; explain workflow and assumptions.
AI tool disclosure	What was AI-assisted, what was not, and how.
Optional: core tradeoff / style guide	Up to one page/diagram for tradeoff; style guide optional.

Official Designathon judging

CRITERION	WEIGHT
Problem framing	25%
Understanding of user context	20%
Degradation screen quality	15%
Domain accuracy	10%
Scope and prioritization	15%
Visual and interaction consistency	15%

Submission: one design file with distinct pages exported using TeamName_Designathon as base filename, zip as TeamName_Designathon.zip; shareable prototype and unlisted demo links. If the Hackathon changes materially, explain departures in its README.

11 Hackathon — what the engineering team must deliver

OFFICIAL SOURCE · Deadline: Sun 4 Oct 2026, 11:59 PM SLST. Booklet pp. 12–13.

Functional requirements

- A responsive WEB application; all four official roles must complete the ordering/planning/loading/delivery/receipt workflow. Native apps are optional, not a substitute.
- Validate weight and volume, temperature compatibility, outlet access, delivery windows and fuel quota constraints.
- Assign orders to vehicles and trips through automatic, assisted or manual-with-validation planning; demonstrate capacity shortage and explicit deferrals.
- Operate and recover gracefully during field connectivity loss, particularly in Driver use cases.
- Seed shared competition reference data and at least one realistic delivery day for a clean judge walkthrough.

Official hand-in

DELIVERABLE	NON-NEGOTIABLE DETAIL
Public deployed URL	Keep deployment live through review and later rounds if selected.
Seeded role credentials	Four accounts: Dispatcher, Loader, Driver, Store Manager.
Monorepo	GitHub repository named TeamName_SolutionName.
Root setup	README, docker-compose.yml or equivalent Compose file, and .env.example. "docker compose up" starts stack, database and seed.
docs folder	Architecture diagram, data model and AI-assisted-work disclosure.
README judge walkthrough	Numbered cross-role steps from planning to completed delivery; setup, credentials, configuration and documented design departures.
Unlisted demo	5–8 minutes: all roles in sequence, then concise code/architecture explanation.

Official Hackathon judging

CRITERION	WEIGHT
Functional completeness across four roles	20%
Planning and allocation engine	20%
Degradation, offline operation and recovery	10%
Fidelity to Day 5 design	10%
Engineering quality and architecture	25%
Creativity	5%
Demo video	10%

12 Datathon Task 1 — handling-time and lateness prediction

OFFICIAL SOURCE · Booklet pp. 15–16, 24–27. Datathon is judged separately and live integration is not required.

For each planned test delivery_id, predict pred_service_min (number of handling minutes at its outlet) and pred_late_prob (a probability from 0 to 1 that arrival is after the outlet window closes). These are TWO different targets: regression and classification/probability estimation.

Construct targets from historical route events

The labels are not supplied as ready-made columns. For every dispatched training order, match exactly one route leg using (route_id, seq_in_route) in deliveries_train.csv and (route_id, seq) in route_legs_train.csv. Validate cardinality before training.

OFFICIAL LABEL DEFINITIONS

Service start = later of actual outlet arrival and the outlet window opening. Service minutes = leave_outlet_time minus service start. Late flag = 1 only when actual arrival is strictly after window_close_time; otherwise 0. An early vehicle waits, and a late arrival is still delivered in the supplied scenario.

WORKED EXAMPLE	TIME	INTERPRETATION
Actual arrival	07:10	Before receiving window
Window opening	07:30	Handling can begin now
Leave outlet	07:50	Handling finished
Service label	20 minutes	07:50 minus 07:30
Window close	08:00	Arrival at 07:10 is not late; label = 0

Prediction-time inputs and modelling process

1. Explore order/train and matching route-leg files; inspect dates, missing values, dispatch_status and match rate.
2. Create labels only from historical actual journey records; handle time in Asia/Colombo consistently.
3. Use available planned/known features: brand, dock type, weight/volume/units, planned travel and arrival, stop sequence, calendar, traffic and road context, if valid.
4. Avoid leakage: actual future arrival, travel or leave time must never be test-time features.
5. Validate on later historical dates; compare baseline median service time and constant/group late rates with locally trained models.
6. Evaluate service regression (e.g., MAE/RMSE) and probability quality (e.g., log loss/Brier + calibration). The booklet does not prescribe exact hidden metrics.
7. Generate exact submission_task1.csv by filling official template; preserve every delivery_id and its original row order.

REQUIRED OUTPUT COLUMNS	VALIDATION
delivery_id	Existing supplied ID, unchanged, once and in original order.
pred_service_min	Finite numeric predicted minutes; investigate impossible negative outputs.
pred_late_prob	Finite numeric probability within [0, 1].

13 Datathon Task 2A — ten-week demand forecast

OFFICIAL SOURCE · Booklet pp. 16–17 and 24–29.

Predict order volume for each supplied depot + brand + forecast-week combination: pred_total_volume_m3 and pred_chilled_volume_m3. Do not convert volume into the number of vehicles or drivers; that is not requested.

The three easy-to-miss rules

8. Build the demand history from BOTH deliveries_train.csv and task1_test_inputs.csv. Count each unique order once, including deferred and not_run orders.
9. Assign demand to the store's requested order_date, not its later dispatch_date. Join calendar.csv and use iso_year + iso_week.
10. Only Fresh has chilled demand. Set predicted chilled volume EXACTLY zero for Style and Tech.

REAL-WORLD EXAMPLE

An outlet requests 5 m³ of Fresh goods in ISO week 38 but delivery is deferred until week 39. Task 2A still records that 5 m³ in week 38 demand. The company needs to forecast what customers requested, not only what the fleet succeeded in carrying.

Recommended modelling path

11. Aggregate order_volume_m3 by depot + brand + ISO year/week. Calculate Fresh chilled subset separately.
12. Build a full weekly calendar and verify missing operating weeks are interpreted correctly (do not equate absent rows with zero without checking).
13. Compare last-week, seasonal and recent rolling-mean baselines with calendar-aware models using only past-known features.
14. Validate chronologically using later weeks; for a 10-week horizon use a strategy that does not borrow unknown future actual demand.
15. Map predictions back onto the provided task2a template row_id; ensure nonnegative outputs and chilled ≤ total.

REQUIRED OUTPUT	CHECK
row_id	Official supplied row identifier, unchanged.

REQUIRED OUTPUT	CHECK
pred_total_volume_m3	Predicted total ordered volume in cubic metres.
pred_chilled_volume_m3	Predicted chilled subset; 0 for Style/Tech and not greater than total.

14 Datathon Task 2B — peak-day allocation (S1)

OFFICIAL SOURCE · Booklet pp. 18–21, 31. This is a constraint-solving task: NO trained model is required.

For a festival-approaching dispatch day at Peliyagoda, allocate every official order as served or deferred. Workshop vehicles cannot be used. Every served order gets one available vehicle_id and trip_id (1 or 2); deferred orders have blank vehicle/trip fields. Key allocations by order_ref, not outlet_id, because an outlet can have multiple orders.

Exact official feasibility rules

#	HARD RULE	PLAIN ENGLISH
1	Same brand and district per trip	Orders sharing vehicle_id + trip_id belong to one brand and district.
2	Chilled → reefer	Reefer may carry ambient too; ambient cannot carry chilled.
3	van_only → van	Do not send trucks to van-only outlets.
4	Home depot	Vehicle serves only outlets from its assigned depot.
5	Whole order	Never split an individual served order across vehicles or trips.
6	Weight AND volume	Trip totals must fit both kg and m ³ vehicle capacity.
7	Max two trips + time budgets	Each vehicle ≤2 trips. Fresh trip minutes combined ≤270; Style + Tech combined ≤480.

Exact Task 2B trip-time formula

Trip duration = depot-to-district free-flow minutes once + (number of orders – 1) × inter-stop free-flow minutes + the sum of service_allowance_min for each order using its brand and dock_type. The time budgets already allow for return travel: DO NOT add a separate return leg.

OFFICIAL BOOKLET EXAMPLE — 3 FRESH STOPS IN GAMPAHA	MINUTES
Depot → Gampaha	37
Two between-stop movements: 2×9	18
Three handling allowances: $15 + 15 + 16$	46
Total trip time	101

Fresh window: 03:30–08:00, total 270 minutes per vehicle. Style and Tech share a 480-minute trading-day budget. A vehicle can mix one Fresh and one Style/Tech trip if compatible and within respective budgets, but never exceed two trips overall.

Allocation method and one-page justification (proposed)

- Filter vehicles to status=available and matching depot; inspect reefer van scarcity and other bottlenecks.
- Determine each order's compatible vehicle set; reserve scarce configurations only when justified.
- Create feasible trip groups by brand + district; check whole-order weight/volume and the exact trip-time formula.
- Choose and document a transparent priority policy considering urgency and previous deferral (deferred_yesterday, days_since_last_served).
- Record every served/deferred outcome and identify whether a deferral was unavoidable from resource shortage or a deliberate tradeoff.
- Run an independent rule audit and official check_allocation.py. The script confirms feasibility, NOT optimality or quality of the deferral policy.

OFFICIAL CSV COLUMNS	REQUIREMENT
scenario, order_ref, outlet_id	Preserve original values; scenario always S1.
decision	served or deferred for each row; replace ALL placeholders.

OFFICIAL CSV COLUMNS	REQUIREMENT
vehicle_id, trip_id	Required for served orders; blank for deferred; trip_id is 1 or 2.

DIFFERENT RULE SETS MATTER

The overall Hackathon planner must account for fuel quotas and outlet windows (Booklet p. 12). Task 2B is judged against its explicitly listed seven scenario rules and budget formula. Keep shared concepts aligned, but never quietly rewrite the Task 2B validator.

15 Datathon data files, rules and deliverables

15.1 Official input map

DATA CATEGORY	FILES / PURPOSE
Training	deliveries_train.csv (one order per delivery_id); route_legs_train.csv (one route leg with actual timings).
Task 1 test	task1_test_inputs.csv; route_legs_test.csv (planned leg information, no actual times).
Task 2A test	task2a_test_inputs.csv (depot + brand + future ISO week and row_id).
Task 2B test	task2b_peak_day_scenarios.csv (orders/access/windows/size/history); task2b_peak_day_fleet.csv (availability).
General reference	outlets.csv, vehicles.csv, calendar.csv, district_travel.csv, service_allowance.csv, traffic_speed.csv, road_conditions.csv.
Templates / check	submission_task1.csv, submission_task2a.csv, submission_task2b.csv; check_allocation.py.

15.2 Official model and data restrictions

- No pretrained models, except the stated synthetic-data-generation or preprocessing exceptions.
- No proprietary API-based modelling/preprocessing; no low-code/no-code AI tools or fully automated end-to-end modelling tools.
- Use provided datasets only for this competition; do not publicly or privately distribute datasets or their derivatives to third parties, or publish them without authorization.
- Preserve an accurate disclosure of AI-assisted work. If any intended workflow is ambiguous, ask the organizer before uploading data or using a tool.

OFFICIAL SOURCE · Booklet p. 22. These rules also apply to collaboration; share documentation and methods internally without exposing restricted data to external tools.

15.3 Complete official Datathon hand-in

ITEM	REQUIRED CONTENT
Architecture diagrams	High-level preprocessing, models and proposed deployment approach.
Preprocessing document	Data preparation, label construction, cleaning, feature engineering and why.
Saved model files	Final models alongside notebook.
Notebook	TeamName_FinalNotebook.ipynb; preserve training/evaluation cells; final cell LOADS saved models and prints Task 1/2A inference inputs/predictions.
Output CSVs	submission_task1.csv, submission_task2a.csv, submission_task2b.csv, exact supplied IDs/columns.
Task 2B policy	Approx. one page or less with calculations, limiting resources, deferrals and cost/impact.
Video / AI disclosure	Unlisted 3–5-minute demo; explain model architecture, preprocessing, labels and challenges; full AI-use disclosure.
Zip and deadline	Compress folder as TeamName_Datathon.zip; deadline 9 Oct 2026 at 11:59 PM Sri Lanka time.

Official Datathon judging

CRITERION	WEIGHT
Data wrangling and label construction	20%
Model and architecture implementation	25%

CRITERION	WEIGHT
Prediction performance: Tasks 1 + 2A	20%
Task 2B feasibility and prioritization policy	15%
Creativity	10%
Demo video	10%

D | Team execution, evidence and quality

16 Integration between the phases — align, do not block

Use common outlet_id, vehicle_id, depots, brand names, calendar and operational definitions everywhere. Designathon specifies the cross-role flow; Hackathon implements and seeds it; Datathon produces separately judged outputs from supplied data. The Datathon result may later enhance planning views but is not an official Hackathon dependency.

SHARED CONTRACT	TEAM HANDOFF
Core entities / IDs	Agree on official field and identifier meanings, especially two orders from one Fresh outlet and delivery_id vs route legs.
Planning constraints	Agree on general rules and keep separate Task 2B scenario validator specification.
State semantics	Distinguish planned ETA from actual arrival and local save from server confirmation.
AI outputs (optional app connection)	Define pred_service_min, pred_late_prob, depot/brand/week total/chilled volume and model version. Never present prediction as an actual event.
Changes after Designathon	Document meaningful changes and reasons in Hackathon README.
Data safety	Do not commit official Datathon CSVs/models-derived private records to a public repo without permission.

17 Execution timeline and clear ownership

WHEN	DESIGN / PRODUCT	ENGINEERING	DATA SCIENCE
28–29 Sep	Finalize role flows, failure screen, clickable prototype, disclosure and video.	Confirm data models, seed format, architecture and feasibility requirements.	Audit files, match route legs, define Task 1 labels and leakage rules.
30 Sep–1 Oct	Support handoff and document changes.	Implement shared data, permissions and end-to-end core order/planning screens.	Build and validate Task 1 baseline; aggregate historical Task 2A weekly demand.
2–3 Oct	Check screen fidelity and failure recovery.	Build loader/driver/store connected flows, offline queue and tested allocation.	Develop Task 2A baselines; start Task 2B compatible vehicle grouping.
4 Oct	Review final user journey and demo.	Complete seeded judge walkthrough, Compose/deploy, 5–8-min demo and submit.	Run initial Task 2B allocation and official checker.
5–7 Oct	Optional insights presentation review.	Support optional integration without jeopardizing submitted build.	Improve and compare models; refine policy, rerun independent checks; save final artifacts.
8–9 Oct	Help with Datathon story/video.	Optional support for local environment and diagrams.	Clean-run notebook, exact templates, final inference cell, files, AI disclosure, 3–5-min demo, zip and submit.

Team ownership recommendation: one accountable product lead, one design lead, one engineering/integration lead and one Datathon owner (Divejikan); smaller teams can combine roles. Agree on a daily “source of truth” decision log rather than passing conflicting screen or API versions.

18 Acceptance tests and judge demonstrations

18.1 Must-pass app walkthrough (proposed acceptance)

22. Log in as Store Manager; place and confirm a realistic order before cutoff. Confirm acknowledgment and correct visibility.
23. Log in as Dispatcher; import/see confirmed queue, allocate feasible trips and explicitly defer at least one order with a reason.
24. Log in as Loader; verify current stop/load order and record a shortage or damage event before departure.
25. Log in as Driver on phone viewport; perform a stop and record proof of delivery. Repeat with connection disabled; show pending queue then recovery.
26. Log in as Store Manager again; confirm receipt and see correct delivery or issue state. Reopen as Dispatcher and inspect the history.
27. Attempt invalid allocations: chilled on ambient, van-only on truck, overweight/over-volume, wrong depot, unsuitable window and quota. Confirm clear rejection.

18.2 Datathon end-to-end checks

CHECK	EVIDENCE TO SHOW
Route join	Each dispatched order maps to exactly one appropriate training route leg.
Labels	Early waiting excluded from service; late flag uses actual ARRIVAL after close, not finish time.
Leakage	Only planned/known features in validation inference; later weeks held out chronologically.
Task 1 CSV	Same IDs and row order as official template; finite service minutes and probabilities in [0, 1].
Task 2A CSV	Same row_id values; every order counted once in requested week; Style/Tech chilled = 0.
Task 2B CSV	Every order_ref once; no placeholders; checker passes; each group obeys brand/district/capacity/time constraints.
Reproducibility	Final notebook runs with documented private local inputs; loads saved files in last cell and demonstrates inference.

18.3 Suggested demo scripts

PHASE	NARRATIVE
Designathon · 3–5 min	Problem and user context → four connected role flows → one fully designed degradation scene → key choices and assumptions.
Hackathon · 5–8 min	Show seeded order → dispatcher plan and deferral → loader update → offline driver outcome and sync → store receipt → code/architecture.
Datathon · 3–5 min	Explain business prediction need → join/label logic → local preprocessing and models → unseen validation → Task 2B feasibility/policy → final inference and files.

19 Risks, tradeoffs and scope control

RISK	PRACTICAL CONTROL
Extra screens displace mandatory journeys	Freeze a minimum core; only add Passport/Detective after judge workflow works.
Incorrect “four roles” interpretation	Use exact official roles; account admin as internal supporting function, not a replacement role.
App promises unsupported real-time visibility	Clearly separate planned, reported, synced and actual evidence; no fake GPS.
Overly clever planner violates limits	Implement transparent validator before any optimization; test all constraints.
Offline demo loses/duplicates events	Use persistent event IDs, visible queue state, idempotent server processing and documented conflict behavior.
Wrong Task 1 labels	Unit-test waiting and late-arrival edge cases; review one sample manually.
Task 2A demand undercounted	Include deferred/not_run and Task1 test orders; group by requested ISO week.
Task 2B uses wrong rule set	Treat its explicit seven checks and formula separately from full app constraints.
Competition data leaks	Private local files and ignore rules; avoid third-party API uploads and public datasets.

RISK	PRACTICAL CONTROL
Setup fails for judge	Start Compose from a clean checkout; seed four accounts, demo day and exact README walkthrough.

20 Team sign-off: decisions to settle immediately

DECISION	RESPONSIBLE / WHAT TO CONFIRM
Master product scope	Agree the minimum four-role journey, core screens and one degradation screen before Designathon submit.
Planning mode	Choose manual validated, assisted or automated assignment for Hackathon, with documented feasibility checks.
Offline boundary	Specify what Driver can record offline and how local/remote conflicts are resolved.
Data schema/API contract	Freeze stable identifiers and field semantics before UI/API integration.
Task 1 label tests	Data owner signs off waiting-window and late-arrival construction with examples.
Task 2A forecasting horizon	Data owner confirms training history, 10-week output coverage and original-demand accounting.
Task 2B policy	Data owner writes scarcity rationale and validates the official CSV with check_allocation.py.
Submission ownership	Name who exports files, runs clean environment test, uploads videos and checks all deadlines.

Appendices

Appendix A | Fast final-delivery checklist

Designathon

- ☐ Four correct personas; connected screen flows and paragraph per screen.
- ☐ At least one fully designed named degradation screen with rationale.
- ☐ High-fidelity prototype and 3–5-minute unlisted video.
- ☐ AI disclosure; optional tradeoff page/style guide; correct zip and links.

Hackathon

- ☐ Responsive four-role web journey, loader/driver phone-size checked.
- ☐ Allocation validates real-world operating constraints; surplus demand yields justified deferrals.
- ☐ Offline capture, sync and recovery demonstrated; shared data and realistic demo day seeded.
- ☐ Four seeded credentials, public URL, GitHub monorepo, README walkthrough, docs and disclosure.
- ☐ docker compose up starts complete database-backed seeded stack; 5–8-minute unlisted video.

Datathon

- ☐ Correct Task 1 historical labels and no actual-future input leakage.
- ☐ Task 1 IDs/order intact; Task 2A row IDs intact and chilled zeros for Style/Tech.
- ☐ Task 2B valid every row and official checker passes; short policy with calculations.
- ☐ Saved models, reproducible final notebook/inference, documentation/diagrams and AI disclosure.
- ☐ 3–5-minute video and TeamName_Datathon.zip ready by 9 Oct 2026.

Appendix B | Working glossary in everyday English

TERM	SIMPLE MEANING
Order vs trip	An order is what an outlet requests; a trip is a vehicle's run carrying one or more served orders.
Depot	The distribution base a vehicle belongs to and from which orders are served.
Delivery window	The time interval an outlet accepts arrivals.
Reefer	A refrigerated vehicle suitable for chilled goods.
Deferral	An order not served in this run; it must keep a reason and not disappear from demand.
Proof of delivery (PoD)	Recorded evidence of a stop outcome, rather than a driver's memory.
Data join key	Column(s) used to connect rows across files, like route ID + stop sequence.
Label	The true historical answer a model learns to predict.

TERM	SIMPLE MEANING
Feature	Known input used by a model, such as size, dock type or planned time margin.
Leakage	Using future answers to predict the future.
Regression	Predicting a number such as handling minutes or cubic metres.
Classification / probability	Estimating a late/not-late outcome and calibrated chance.
ISO week	Standard year + week grouping in calendar.csv.
Hard constraint	A rule that must never be broken, even for a high-priority order.
Feasible vs optimal	Feasible follows all rules; optimal additionally meets some chosen objective as strongly as possible.
Synchronization	Sending locally saved field changes to the server once connectivity returns.

Appendix C | Official-source and recommendation map

SOURCE PAGES	USE IN THIS MASTER PLAN
Booklet pp. 3–7	Theme, Waypoint scenario, roles, constraints, objective and common data.
Booklet pp. 9–10	Designathon required deliverables, judging, naming and submission.
Booklet pp. 12–13	Hackathon requirements, deployment/repo/Compose/seed, judging and video.
Booklet pp. 15–17	Task 1 label/prediction definition, Task 2A ten-week volume accounting and templates.
Booklet pp. 18–21	Task 2B scenario, exact seven rules, time formula/budgets and deferral policy.
Booklet pp. 22–23	Datathon restrictions, mandatory artifacts, judging and deadline.
Booklet pp. 24–31	File relationships, reference fields and official allocation checker.
WayLoom recommendations	Feature package, suggested screen map, visual identity, schemas, stack, model choices, tests and work ownership are team proposals—not official organizer mandates.

FINAL SOURCE-OF-TRUTH REMINDER

This plan is a human-readable working interpretation of the official booklet, not a substitute for it. If any issue conflicts with the organizer's booklet or later clarification, use the official requirement and update the team decision log.

Prepared as an internal team-shareable WayLoom document. Names and worked IDs in explanations are illustrative. Source: Rootcode Tech-Triathlon 2026 Challenge Booklet (33 pages), supplied in this project.